

SuperBodega

Sistema de E-Commerce

Documentación Técnica del Proyecto Final

Programación III | Universidad Mariano Gálvez de Guatemala | 2026

Integrante	Responsabilidad
Gustavo Ramírez	Arquitectura, Docker y Base de Datos
Katherine Daniela Portillo Melendez	API Admin — Categorías, Productos y Proveedores
Jorge Danilo Ucelo Román	API Admin — Clientes, Compras, Ventas y Reportes
Ariatna Odeth Berganza Villatoro	API E-Commerce, Email e Interfaces
Jonathan Estuardo Dionicio Orellana	RabbitMQ, Versión Asíncrona y Pruebas K6

1. Descripción General del Proyecto

SuperBodega es un sistema completo de comercio electrónico desarrollado con .NET Core 9. Reemplaza una plataforma antigua en PHP que presentaba caídas frecuentes y pérdida de pedidos, generando pérdidas monetarias al negocio.

1.1 Problema que Resuelve

- La plataforma anterior en PHP presentaba caídas constantes bajo alta demanda.
- Se perdían pedidos frecuentemente, generando pérdidas monetarias.
- La tecnología antigua no permitía escalar el sistema.
- No existía un sistema de notificaciones para los clientes.

1.2 Solución Implementada

- Dos APIs RESTful modernas con .NET Core 9.
- Base de datos SQL Server en contenedor Docker con volumen persistente.
- Sistema de colas con RabbitMQ para procesamiento asíncrono.
- Notificaciones por email automáticas usando Gmail SMTP.
- Interfaces de usuario HTML para administración y tienda en línea.
- Pruebas de carga con K6 — 0% errores con 50 usuarios concurrentes.

2. Arquitectura del Sistema

El sistema sigue una arquitectura en capas (Clean Architecture) con separación clara de responsabilidades.

2.1 Proyectos de la Solución

Proyecto	Tipo	Puerto	Descripción
SuperBodega.API	Web API	5070	API de administración del negocio
SuperBodega.ECommerce	Web API	5225	API pública para la tienda
SuperBodega.Core	Class Library	—	Entidades del dominio e interfaces
SuperBodega.Infrastructure	Class Library	—	Acceso a datos, mensajería y servicios
SuperBodega.Tests	xUnit Test	—	Proyecto de pruebas unitarias

2.2 Infraestructura Docker

Servicio	Imagen	Puerto	Propósito
SQL Server	mssql/server:2022-latest	1434	Base de datos principal
RabbitMQ	rabbitmq:3-management	5672/15672	Broker de mensajes

2.3 Tecnologías Utilizadas

Tecnología	Versión	Uso
C# / ASP.NET Core	9.0	Lenguaje principal y framework de las APIs
Entity Framework Core	9.0.4	ORM para acceso a base de datos
SQL Server	2022 (Docker)	Motor de base de datos relacional
RabbitMQ	3.13 (Docker)	Broker de mensajes para colas asíncronas
MailKit	4.8.0	Librería para envío de emails via Gmail SMTP
Newtonsoft.Json	9.0.4	Serialización JSON con manejo de ciclos
Swashbuckle/Swagger	6.9.0	Documentación automática de APIs
Grafana K6	1.7.1	Framework de pruebas de carga
HTML/CSS/JavaScript	Puro	Interfaces de usuario sin frameworks

3. Modelo de Base de Datos

La base de datos SuperBodegaDB corre en SQL Server 2022 dentro de un contenedor Docker con volumen persistente. Las migraciones se gestionan con Entity Framework Core.

3.1 Entidades del Dominio

Entidad	Tabla en BD	Campos Principales
Categoria	Categorias	Id, Nombre, Descripción, Activo
Producto	Productos	Id, Código, Nombre, Precio, Stock, StockMinimo, CategoricalId
Cliente	Clientes	Id, Nombre, Apellido, Email, Teléfono, Dirección
Proveedor	Proveedores	Id, Nombre, Contacto, Email, Teléfono
Compra	Compras	Id, NumeroFactura, Fecha, Total, Estado, ProveedorId
DetalleCompra	DetallesCompra	Id, Cantidad, PrecioUnitario, Subtotal, CompraId, ProductId
Venta	Ventas	Id, NumeroOrden, Fecha, Total, Estado, ClienteId
DetalleVenta	DetallesVenta	Id, Cantidad, PrecioUnitario, Subtotal, VentaId, ProductId

3.2 Manejo de Inventario

- Compra registrada: el stock del producto aumenta según la cantidad comprada.
- Venta realizada: el stock del producto disminuye según la cantidad vendida.
- Validación de stock: si no hay stock suficiente la venta es rechazada con error 400.
- Transacciones de BD: cada operación usa transacciones para garantizar consistencia.

3.3 Aplicar Migraciones

```
dotnet ef database update --project src/SuperBodega.Infrastructure --startup-project src/SuperBodega.API
```

4. API Principal de Administración — Puerto 5070

Swagger disponible en: <http://localhost:5070/swagger>

4.1 Endpoints Disponibles

Módulo	Método	Endpoint	Descripción
Categorías	GET	/api/Categorias	Listar todas las categorías
Categorías	POST	/api/Categorias	Crear nueva categoría
Categorías	PUT	/api/Categorias/{id}	Actualizar categoría
Categorías	DELETE	/api/Categorias/{id}	Eliminar categoría
Productos	GET	/api/Productos	Listar productos con categoría
Productos	POST	/api/Productos	Crear producto con DTO
Proveedores	GET/POST	/api/Proveedores	CRUD de proveedores
Clientes	GET/POST	/api/Clientes	CRUD de clientes
Compras	POST	/api/Compras	Registrar compra — sube stock
Ventas	POST	/api/Ventas	Crear venta síncrona (201)
Ventas	PATCH	/api/Ventas/{id}/estado	Cambiar estado de venta
Reportes	GET	/api/Reportes/ventas/producto	Ventas por producto
Reportes	GET	/api/Reportes/ventas/cliente	Ventas por cliente
Reportes	GET	/api/Reportes/ventas/proveedor	Compras por proveedor
Ventas Async	POST	/api/async/VentasAsync	Crear venta asíncrona (202)

4.2 Estados de una Venta

Estado	Descripción	Email automático
Pendiente	Venta recién creada	Sí — Pedido recibido
Despachado	Pedido en camino al cliente	Sí — Pedido despachado
Entregado	Pedido entregado exitosamente	Sí — Pedido entregado
Cancelado	Venta cancelada	No

5. API E-Commerce — Puerto 5225

Swagger disponible en: <http://localhost:5225/swagger>

5.1 Catálogo de Productos

Parámetro	Tipo	Default	Descripción
pagina	int	1	Número de página para paginación
tamano	int	10	Cantidad de productos por página
categoriald	int?	null	Filtrar por categoría específica
busqueda	string?	null	Buscar por nombre o descripción

5.2 Flujo de Compra — POST /api/Carrito/realizar-compra

- Valida que el cliente exista en el sistema.
- Verifica stock disponible para cada producto.
- Crea la venta y sus detalles en una transacción de base de datos.
- Actualiza el inventario de cada producto.
- Publica un mensaje en RabbitMQ para enviar notificación por email.
- Retorna el número de orden y total de la compra.

6. Sistema de Mensajería Asíncrona

RabbitMQ desacopla el procesamiento de pedidos de las notificaciones por email y permite la versión asíncrona de ventas.

6.1 Colas Implementadas

Cola	Productor	Consumidor	Propósito
notificaciones-pedidos	CarritoController, VentasController	NotificacionConsumerService	Envío de emails
ventas-async	VentasAsyncController	VentaAsyncWorker	Ventas en segundo plano

6.2 Versión Síncrona vs Asíncrona

Aspecto	Versión Síncrona	Versión Asíncrona
Endpoint	POST /api/Ventas	POST /api/async/VentasAsync
Respuesta HTTP	201 Created con datos de venta	202 Accepted inmediatamente
Procesamiento	Directo en el request HTTP	En segundo plano via RabbitMQ
Tiempo promedio	3.62 ms	14.86 ms
Validación de stock	En el Controller	En el Worker en segundo plano

6.3 Notificaciones por Email

- Pedido recibido: al crear la venta con estado Pendiente.
- Pedido despachado: al cambiar el estado a Despachado.
- Pedido entregado: al cambiar el estado a Entregado.

Las credenciales de Gmail se leen del appsettings.json que está excluido del repositorio por seguridad.

7. Librerías Utilizadas

Librería	Versión	Propósito
Entity Framework Core	9.0.4	ORM para mapear clases C# a tablas SQL Server
EFCore.SqlServer	9.0.4	Proveedor de SQL Server para EF Core
Newtonsoft.Json	9.0.4	Serialización JSON con ReferenceLoopHandling.Ignore
Swashbuckle.AspNetCore	6.9.0	Generador automático de documentación Swagger
RabbitMQ.Client	Latest	Cliente oficial de RabbitMQ para .NET
MailKit	4.8.0	Envío de emails via Gmail SMTP
MimeKit	4.8.0	Construcción de mensajes de email en formato HTML
Microsoft.Extensions.Hosting	9.0	BackgroundService para servicios en segundo plano

8. Pruebas

8.1 Scripts de Carga K6

Script	Descripción	Comando
load-test.js	50 VUs por 2 minutos	k6 run tests/k6/load-test.js
sinrono-vs-asincrono.js	Comparación de tiempos	k6 run tests/k6/sinrono-vs-asincrono.js
ventas-sync-vs-async.js	Ventas sync vs async	k6 run tests/k6/ventas-sync-vs-async.js

8.2 Resultados

Métrica	Prueba General	Síncrono	Asíncrono
Requests totales	10,728	300	300
Tasa de error	0%	0%	0%
Tiempo promedio	2.33 ms	3.62 ms	14.86 ms
P95	3.49 ms	4.57 ms	52.46 ms
Usuarios concurrentes	50 VUs	20 VUs	10 VUs

9. Guía de Instalación

9.1 Requisitos Previos

Software	Versión	Propósito
.NET SDK	9.0	Compilar y ejecutar las APIs
Docker Desktop	Reciente	Contenedores de SQL Server y RabbitMQ
Git	Cualquiera	Clonar el repositorio
K6	v1.0+	Pruebas de carga
Postman	Reciente	Probar endpoints manualmente

9.2 Pasos de Instalación

1. Clonar el repositorio:

```
git clone https://github.com/GussRamirez0/SuperBodega-Final.git
```

2. Levantar Docker:

```
docker-compose up -d
```

3. Copiar configuración:

```
Copy-Item src/SuperBodega.API/appsettings.example.json  
src/SuperBodega.API/appsettings.json
```

4. Editar appsettings.json con credenciales de Gmail.

5. Aplicar migraciones:

```
dotnet ef database update --project src/SuperBodega.Infrastructure --startup-  
project src/SuperBodega.API
```

6. Levantar API Admin (Terminal 1):

```
dotnet run --project src/SuperBodega.API
```

7. Levantar API E-Commerce (Terminal 2):

```
dotnet run --project src/SuperBodega.ECommerce
```

9.3 URLs del Sistema

Servicio	URL	Credenciales
Swagger API Admin	http://localhost:5070/swagger	—
Swagger E-Commerce	http://localhost:5225/swagger	—
Interfaz Admin	src/SuperBodega.Web/wwwroot/index.html	—
Tienda E-Commerce	src/SuperBodega.Web/wwwroot/ecommerce.html	—
RabbitMQ Panel	http://localhost:15672	admin / admin123
SQL Server	localhost,1434	sa / SuperBodega2024!

10. Repositorio GitHub

<https://github.com/GussRamirez0/SuperBodega-Final>

10.1 Ramas del Proyecto

Rama	Integrante	Commits
main	Todos (merge final)	Código completo integrado
feature/arquitectura-docker-bd	Gustavo Ramírez	10 commits
feature/katherine-categorias-productos-proveedores	Katherine Portillo	8 commits
feature/jorge-clientes-compras-ventas-reportes	Jorge Ucelo	10 commits
feature/ariatna-ecommerce-email-interfaces	Ariatna Berganza	11 commits
feature/jonathan-rabbitmq-async-tests	Jonathan Dionicio	10 commits

10.2 Estructura de Carpetas

Carpeta / Archivo	Descripción
src/SuperBodega.API/	API principal de administración con controllers
src/SuperBodega.ECommerce/	API de tienda en línea con catálogo y carrito
src/SuperBodega.Core/	Entidades del dominio e interfaces
src/SuperBodega.Infrastructure/	DbContext, repositorios, mensajería y email
src/SuperBodega.Web/wwwroot/	Interfaces HTML de administración y tienda
tests/SuperBodega.Tests/	Proyecto de pruebas unitarias xUnit
tests/k6/	Scripts de pruebas de carga K6
docker-compose.yml	Configuración de todos los servicios Docker
README.md	Documentación de instalación y uso

11. Lista de Cotejo — Participación por Integrante

Evidencia del trabajo realizado por cada integrante del equipo. Los commits están disponibles en el repositorio GitHub.

11.1 Gustavo Ramírez — Arquitectura, Docker y Base de Datos

#	Tarea Completada	Archivo
1	Estructura de solución con 4 proyectos	SuperBodega.sln
2	docker-compose.yml con SQL Server y RabbitMQ	docker-compose.yml
3	8 entidades del dominio creadas	src/SuperBodega.Core/Entities/
4	4 interfaces de repositorio definidas	src/SuperBodega.Core/Interfaces/
5	ApplicationDbContext con todos los DbSet	src/SuperBodega.Infrastructure/Data/
6	DesignTimeDbContextFactory configurada	src/SuperBodega.Infrastructure/Data/
7	Migraciones EF Core aplicadas	src/SuperBodega.Infrastructure/Migrations/
8	Repositorios base implementados	src/SuperBodega.Infrastructure/Repositories/
9	.gitignore con archivos sensibles excluidos	.gitignore
10	README.md con instrucciones de instalación	README.md

Rama: feature/arquitectura-docker-bd — 10 commits — Mergeado a main ✓

11.2 Katherine Daniela Portillo Melendez — Categorías, Productos y Proveedores

#	Tarea Completada	Archivo
1	CategoriasController con CRUD completo	Controllers/CategoriasController.cs
2	ProductosController con DTO para ciclos JSON	Controllers/ProductosController.cs
3	ProveedoresController con CRUD completo	Controllers/ProveedoresController.cs
4	Filtro de productos por categoría	Controllers/ProductosController.cs
5	Endpoints probados en Swagger sin errores	http://localhost:5070/swagger
6	Commits en rama propia	feature/katherine-categorias-productos-proveedores
7	Merge a main realizado	github.com/GussRamirez0/SuperBodega-Final
8	Código funcional en el sistema final	src/SuperBodega.API/Controllers/

Rama: feature/katherine-categorias-productos-proveedores — 8 commits — Mergeado a main ✓

11.3 Jorge Danilo Ucelo Román — Clientes, Compras, Ventas y Reportes

#	Tarea Completada	Archivo
1	ClientesController con CRUD completo	Controllers/ClientesController.cs
2	ComprasController con aumento de stock	Controllers/ComprasController.cs
3	VentasController con validación de stock	Controllers/VentasController.cs
4	PATCH para cambio de estados de ventas	Controllers/VentasController.cs
5	ReportesController con 4 tipos de reporte	Controllers/ReportesController.cs
6	Transacciones de BD en compras y ventas	Controllers/ComprasController.cs
7	Filtro de ventas por período	Controllers/VentasController.cs
8	Commits en rama propia	feature/jorge-clientes-compras-ventas-reportes
9	Merge a main realizado	github.com/GussRamirez0/SuperBodega-Final
10	Código funcional en el sistema final	src/SuperBodega.API/Controllers/

Rama: feature/jorge-clientes-compras-ventas-reportes — 10 commits — Mergeado a main ✓

11.4 Ariatna Odeth Berganza Villatoro — E-Commerce, Email e Interfaces

#	Tarea Completada	Archivo
1	CatalogoController con paginación	ECommerce/Controllers/CatalogoController.cs
2	Filtro por categoría y búsqueda de texto	ECommerce/Controllers/CatalogoController.cs
3	CarritoController con proceso de compra	ECommerce/Controllers/CarritoController.cs
4	Validación de stock en el carrito	ECommerce/Controllers/CarritoController.cs
5	EmailService con Gmail SMTP via MailKit	Infrastructure/Services/EmailService.cs
6	3 tipos de notificaciones por email	Infrastructure/Services/EmailService.cs
7	index.html interfaz admin funcional	SuperBodega.Web/wwwroot/index.html
8	ecommerce.html tienda funcional	SuperBodega.Web/wwwroot/ecommerce.html
9	CORS habilitado en la API E-Commerce	SuperBodega.ECommerce/Program.cs
10	Commits en rama propia	feature/ariatna-ecommerce-email-interfaces
11	Merge a main realizado	github.com/GussRamirez0/SuperBodega-Final

Rama: feature/ariatna-ecommerce-email-interfaces — 11 commits — Mergeado a main ✓

11.5 Jonathan Estuardo Dionicio Orellana — RabbitMQ, Asíncrono y Pruebas

#	Tarea Completada	Archivo
1	RabbitMQService con publicación de mensajes	Infrastructure/Messaging/RabbitMQService.cs
2	NotificacionConsumerService funcional	Infrastructure/Services/NotificacionConsumerService.cs
3	VentaAsyncWorker procesando en segundo plano	Infrastructure/Services/VentaAsyncWorker.cs
4	VentasAsyncController — responde 202	API/Controllers/VentasAsyncController.cs
5	load-test.js — 10,728 req con 0% errores	tests/k6/load-test.js
6	sincrono-vs-asincrono.js ejecutado	tests/k6/sincrono-vs-asincrono.js
7	ventas-sync-vs-async.js — 100% checks	tests/k6/ventas-sync-vs-async.js
8	Colección Postman documentada	tests/SuperBodega.Tests/
9	Commits en rama propia	feature/jonathan-rabbitmq-async-tests
10	Merge a main realizado	github.com/GussRamirez0/SuperBodega-Final

Rama: feature/jonathan-rabbitmq-async-tests — 10 commits — Mergeado a main ✓

11.6 Resumen de Participación

Integrante	Rama	Commits	Estado
Gustavo Ramírez	feature/arquitectura-docker-bd	10	Mergeado ✓
Katherine Portillo	feature/katherine-categorias-productos-proveedores	8	Mergeado ✓
Jorge Ucelo	feature/jorge-clientes-compras-ventas-reportes	10	Mergeado ✓
Ariatna Berganza	feature/ariatna-ecommerce-email-interfaces	11	Mergeado ✓
Jonathan Dionicio	feature/jonathan-rabbitmq-async-tests	10	Mergeado ✓